

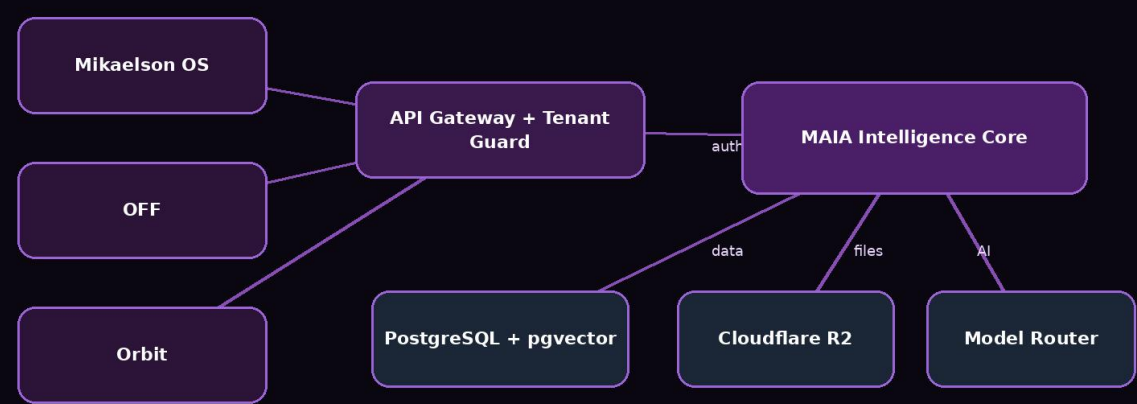
MAIA TECHNICAL ARCHITECTURE

v1 - Engineering Specification

From Blueprint to Build Plan

System Architecture

MAIA Core is a central independent platform connected by secure APIs.



MAIA Intelligence Core | Technical Architecture v1

Purpose

This document translates the MAIA Intelligence Blueprint into a single technical architecture for engineering execution: stack, tenancy, auth, security, data, memory, RAG, model routing, actions, events, notifications, cost, audit, recovery, API contracts, environments, CI/CD, testing and MVP scope.

Prepared for	Date	Classification
MAIA Business / MAIA Intelligence Core	17 June 2026	Confidential

01. Executive Summary

MAIA Intelligence is an independent central platform, not a feature embedded inside Mikaelson, OFF or Orbit. Connected products must call MAIA Core through authenticated APIs and pass tenantId, productContext, user identity and permission context into every request. The goal of this technical architecture is to provide a buildable, secure and scalable engineering specification for the MVP foundation.

- Build MAIA as a central intelligence core with product-specific behavior.
- Start with security, tenancy, permissions and governance before advanced AI automation.
- Use a professional B2B SaaS stack that is learnable, maintainable and expandable.
- Keep all AI calls auditable, cost-tracked and routed through a provider-independent Model Router.

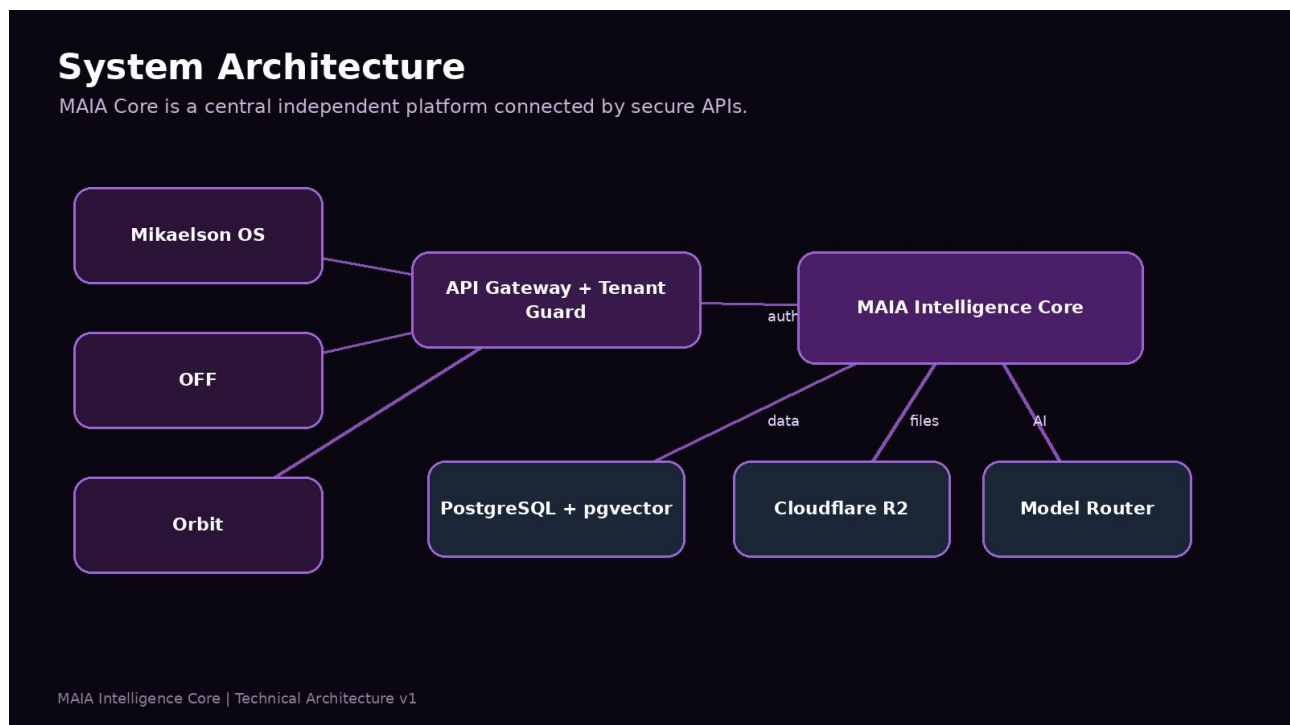
02. Core Principles

These principles are non-negotiable architecture rules. If a future implementation conflicts with them, the implementation is wrong, not the principle. Charming little detail, but systems survive because someone says no to shortcuts.

- Single Intelligence Architecture: MAIA is one central intelligence with multiple product contexts.
- Multi-Tenant First: every organization is isolated by design.
- Security Before Intelligence: no AI response bypasses tenant, role, permission or sensitive access gates.
- Model Independence: application code calls the MAIA Model Router, never a provider directly.
- Event-Driven Awareness: MAIA should detect relevant platform changes and notify users intelligently.
- Auditable AI: important responses and actions must leave evidence without exposing private chain-of-thought.
- Safe Failure: when uncertain, MAIA protects data and blocks critical action.

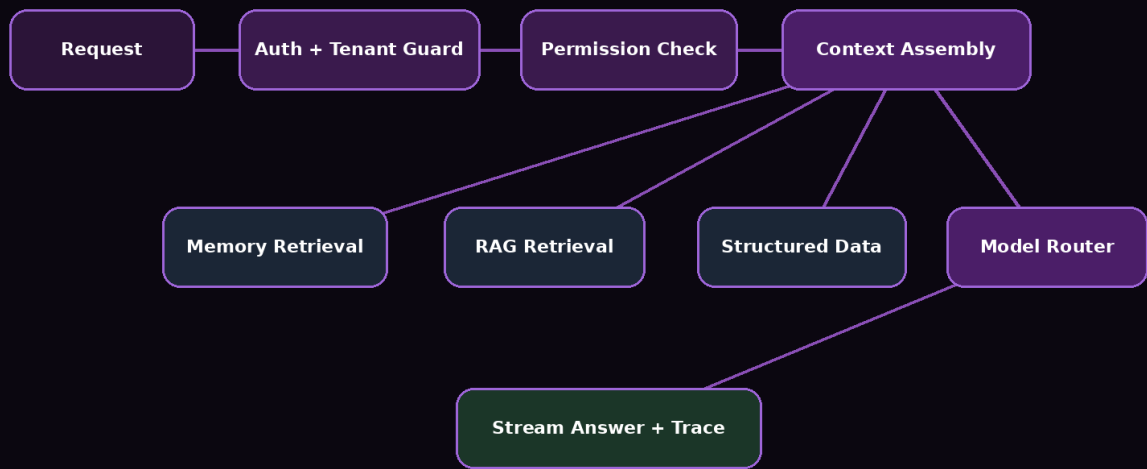
03. System Architecture

The system uses a layered architecture. Product interfaces send requests to MAIA Core. The API layer validates identity, tenant, permissions and product context. The orchestration layer assembles memory, documents, structured data, tools and model policy. The AI layer generates responses through the Model Router. Audit, cost and event services record what happened.



Chat Request Flow

The response pipeline must validate security before reasoning.



MAIA Intelligence Core | Technical Architecture v1

Architecture rule

All product apps are clients of MAIA Core. They pass productContext; they do not own core intelligence logic.

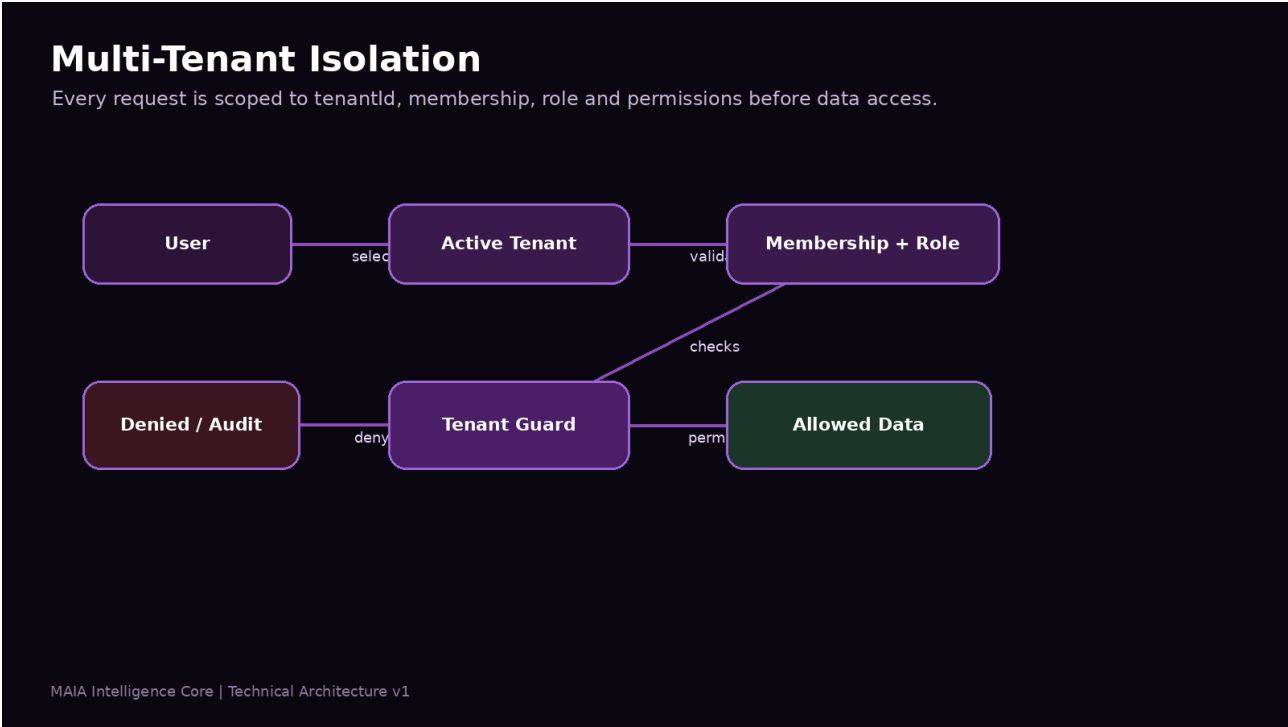
04. Technology Stack

The selected stack balances speed, enterprise readiness and future scalability. The goal is not to collect fashionable tools like trading cards. The goal is to build a system that can survive real users, real data and real failure modes.

Layer	Selected Technology	Use in MAIA
Frontend	Next.js App Router + TypeScript	Dashboard, chat, admin, product shells
Hosting	Vercel	Deployments, previews, production hosting
Database	PostgreSQL on Neon	Relational data, tenant isolation, audit, costs
ORM	Prisma	Schema, migrations, typed database client
Vector Search	pgvector	Document and memory retrieval
Auth	Clerk Organizations	B2B tenants, invitations, sessions, roles
Storage	Cloudflare R2	Private documents, tickets, contracts, PDFs
Queue	Upstash Redis + BullMQ	Background jobs for processing/events
AI	OpenAI first via MAIA Model Router	Streaming, embeddings, reasoning, classification
Monitoring	Sentry + PostHog + OpenTelemetry	Errors, analytics, tracing, observability

05. Multi-Tenant Architecture

Tenant isolation is the backbone of MAIA. A tenant represents an organization/workspace. Every tenant-scoped table must include tenantId. Every query must be filtered by tenantId unless executed through an explicit SUPER_ADMIN path with audit logging.



Principle	Technical Rule
Tenant-scoped by default	Every organization-owned record includes tenantId
Membership-based access	User sees data only with active membership
Role per tenant	User may have different roles in different tenants
No cross-tenant memory	Memory retrieval filters tenantId and security level
No cross-tenant RAG	Chunks inherit document permissions
Super admin exceptional	Allowed only through audited admin flows

06. Authentication & Authorization

Authentication proves who the user is. Authorization decides what the user can do. MAIA needs both, because knowing someone’s email does not magically grant access to client data, despite what many tragic SaaS dashboards seem to imply.

Role	Typical User	Core Access
SUPER_ADMIN	MAIA owner/CTO	Full platform access with mandatory audit
ORG_ADMIN	Client company admin	Manage tenant users, docs, projects, settings
MANAGER	Team lead	Operational read/write
MEMBER	Internal user	Assigned data, chat, tasks
CLIENT	External client	Limited project/customer portal data
VIEWER	Stakeholder	Read-only access
AI_AGENT	System actor	Constrained tool execution

Permission Group	Examples
tenant	tenant.read, tenant.update, tenant.invite
memory	memory.read, memory.write, memory.archive, memory.delete
documents	documents.read, documents.upload, documents.process, documents.delete
projects	projects.read, projects.write, tasks.write, quotes.write
sensitive	sensitive.read, sensitive.write, sensitive.full_access
actions	actions.propose, actions.execute, actions.critical_confirm
admin	admin.audit, admin.costs, admin.incidents
notifications	notifications.read, notifications.manage

Onboarding rule

Admin creates organization -> invites users -> user activates account -> user creates Sensitive Access Code if sensitive modules are enabled.

Capability	SUPER_ADMIN	ORG_ADMIN	MANAGER	MEMBER	CLIENT	VIEWER
Tenant settings	Full	Full own tenant	No	No	No	No
Invite users	Full	Yes	Optional	No	No	No
Read memory	All audited	Tenant allowed	Allowed scope	Assigned scope	Project scope	Read-only scope
Write memory	All audited	Yes	Yes	Limited	No	No
Archive memory	All audited	Yes	Review only	No	No	No
Delete memory	All audited	Restricted	No	No	No	No
Upload documents	All audited	Yes	Yes	Yes	Limited	No
Process documents	All audited	Yes	Yes	No	No	No
Read sensitive data	All audited	With code	With code + role	No by default	Own only + code	No
Execute actions	All audited	Yes	Limited	Limited own scope	No critical	No
View audit/cost	Full	Tenant	Limited	No	No	No

Permission Enforcement Rule

Permissions are enforced server-side.

Frontend visibility is only a user experience layer.

The backend remains the source of truth for authorization decisions.

Every request must validate:

- Tenant scope
 - Membership
 - Role
 - Permission
 - Sensitive access state
- before business logic execution.

07. Security Architecture

Security is implemented as layered controls: tenant isolation, role-based access, permission checks, sensitive sessions, encryption, audit logs, rate limits and safe failure. MAIA must deny access when permissions are unclear.

Control	v1 Decision
Sensitive code	6 digits, hash only using Argon2id preferred
Failed attempts	5 before 15-minute lockout
Sensitive read session	30 minutes or until logout
Sensitive full access	Until logout or 30 minutes inactivity
Critical actions	Always require confirmation
Encryption	Sensitive payloads encrypted server-side
Logging	No raw sensitive data in logs

Security default

If permissions, tenant or sensitive session are unclear, deny access and log the denial.

Threat	Impact	Mitigation
Cross-tenant access	Client data leak between organizations.	TenantGuard on every query, membership check, RLS later, audit failed attempts.
Prompt injection	User/document tries to override system rules.	Instruction hierarchy, tool policy isolation, source labeling, never execute from document text alone.
Sensitive data leakage	RFC, CFDI, contracts or invoices exposed.	Sensitive sessions, masking, encryption, no raw sensitive logs, final confirmation.
Privilege escalation	User gains admin-like access.	RBAC + permission matrix + server-side checks + admin audit.
Webhook spoofing	Fake event triggers action or alert.	HMAC signatures, timestamp window, eventId deduplication.
Replay attack	Old webhook/action replayed.	Idempotency keys, event uniqueness, timestamp validation.
Cost abuse	Excessive AI/doc processing usage.	Rate limits, budget alerts, soft/hard limits, emergency stop.
Data corruption	Bad memory update, duplicate action, failed migration.	Versioning, archive-first, idempotency, migration review, backups.

Security Principle

Threat prevention must occur before model execution.

MAIA never assumes a request is valid because it reached the API.

Every request is considered untrusted until:

- Identity validated
- Tenant validated
- Permissions validated
- Sensitive state validated

08. Data Architecture

The data architecture supports tenant isolation, product contexts, executive memory, secure documents, conversations, tool execution, events, notifications, model usage, costs and auditability.

Entity	Purpose	Critical Fields
Tenant	Workspace boundary	id, name, slug, plan, status
User	Human/system user	id, clerkUserId, name, email, status
Membership	Role inside tenant	tenantId, userId, role, permissions
ProductContext	Mikaelson/OFF/Orbit/MAIA mode	key, tone, enabledTools, limits
Memory	Executive memory item	tenantId, ownerType, type, content, securityLevel
MemoryVersion	History of memory changes	memoryId, previousContent, newContent, reason
SensitiveProfile	Encrypted fiscal/legal profile	tenantId, ownerId, encryptedPayload
SensitiveSession	Temporary sensitive access	userId, tenantId, mode, expiresAt
Document	File metadata	tenantId, ownerId, storageKey, securityLevel
DocumentChunk	RAG searchable text	documentId, tenantId, content, embedding
Conversation	Chat thread	tenantId, userId, productContext
Message	Chat message	conversationId, role, content, securityLevel
Project	Operational project	tenantId, status, risk, budget
Task	Work item	projectId, title, status, priority
ToolLog	Tool execution record	toolName, userId, tenantId, status
Event	System signal	type, tenantId, source, priority
Notification	User alert	userId, severity, recommendation, status
AITrace	Reasoning audit	model, memoryUsed, docsUsed, toolsUsed
UsageLog	Usage metric	tenantId, provider, model, tokens
CostLedger	Cost accounting	tenantId, operation, estimatedCost

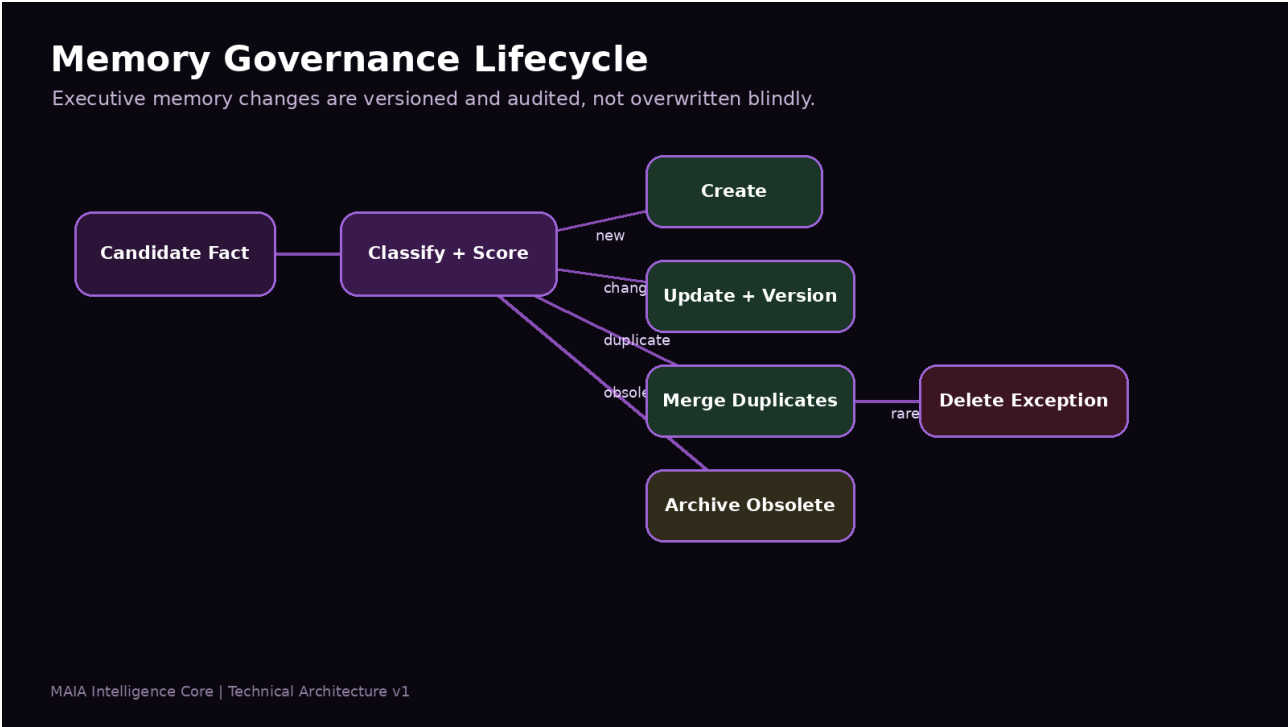


Relationship	Cardinality	Implementation Rule
Tenant -> Membership	1:N	Deleting tenant should be restricted; archive tenant instead.
User -> Membership	1:N	User may belong to multiple tenants with different roles.
Tenant -> Memory	1:N	Every memory is tenant-scoped unless explicitly global.
Memory -> MemoryVersion	1:N	Every update creates a version before changing active memory.
Document -> DocumentChunk	1:N	Chunks inherit tenantId and securityLevel from parent document.
Conversation -> Message	1:N	Messages are not long-term memory by default.
Tool execution -> ToolLog	1:N	Every execution attempt logs status and confirmation.
AI request -> AITrace	1:1	Every response has one trace record.

Prisma schema v1 should include: Tenant, User, Membership, ProductContext, Memory, MemoryVersion, SensitiveProfile, SensitiveSession, Document, DocumentChunk, Conversation, Message, Project, Task, Tool, ToolLog, Event, Notification, AITrace, UsageLog and CostLedger. Each tenant-scoped model must include tenantId and indexed foreign keys.

09. Memory Engine

The Memory Engine is not chat history. It is a curated executive memory system. It stores durable facts, decisions, preferences, risks, strategic context and sensitive records only when they have long-term operational value.



Operation	When It Happens	Guardrail
Create	New relevant fact/preference/decision/risk	Requires owner, type, securityLevel, confidence
Update	Known fact changes	Create MemoryVersion before update
Merge	Duplicates/conflicting records represent same fact	Keep strongest source, preserve history
Archive	Obsolete but historically useful	Remove from default retrieval
Delete	Legal request, user request, unsafe/corrupt data	Requires permission and audit
Review	Low-confidence/sensitive/conflicting memory	Human/admin approval before commit

Memory Evaluation Criteria

The Memory Scoring Engine evaluates whether information should become part of MAIA's long-term memory. Each candidate memory is evaluated across multiple dimensions before storage, review or rejection.

Score	Formula / Rule	Decision
Importance	Business relevance + future usefulness + operational impact.	High: candidate for memory. Low: keep in chat only.
Confidence	Source reliability + explicitness + corroboration.	Low: review queue or user confirmation.
Sensitivity	Detect fiscal/legal/personal/contract terms.	Sensitive: code-gated storage and access.
Recency	Newer facts weigh more unless older fact is still active.	Used in retrieval ranking.
Conflict	Contradicts active memory.	Create MemoryVersion and review if sensitive.

Memory Weight Distribution

Factor	Weight
Importance	35%
Confidence	25%
Sensitivity	20%
Recency	10%
Conflict Detection	10%

Memory Decision Thresholds

Final Score	Action
80–100	Store Automatically
60–79	Store & Review
Below 60	Discard

Memory Lifecycle Rule

Create → Update → Merge → Archive
Permanent deletion is reserved for compliance requirements, user requests or administrative actions.

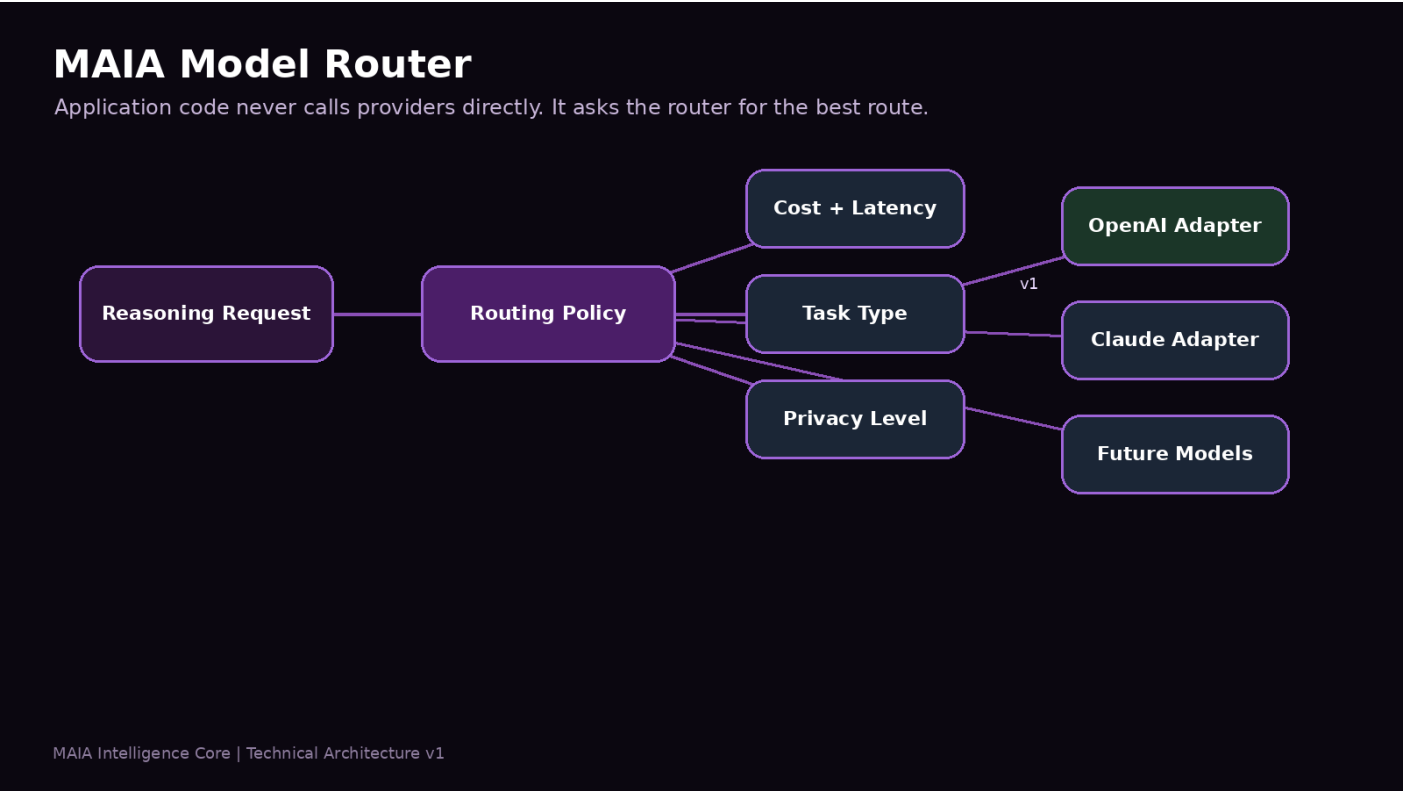
10. Knowledge Engine

The Knowledge Engine handles documents and RAG. Documents are stored privately, processed asynchronously, chunked, embedded and retrieved only after tenant and permission validation.

Step	Technical Behavior
Upload	Store file in R2, create Document, assign tenant/security level
Extract	Extract text asynchronously and store processing status
Chunk	Split text into fragments with page/section metadata
Embed	Generate embeddings and store in DocumentChunk
Retrieve	Run tenant-scoped vector/keyword search
Answer	Pass only allowed chunks into ContextPackage
Trace	AITrace records document/chunk evidence

11. Model Router

The Model Router prevents vendor lock-in, manages cost and routes each AI task to the right provider/model class. OpenAI is the first provider, not the permanent prison.



Task Type	Preferred Model Class	Reason
Simple classification	Small/fast model	Low cost and latency
Embeddings	Embedding model	Memory/document retrieval
Strategic reasoning	Advanced reasoning model	Better synthesis/judgment
Long document analysis	Long-context model	Large evidence windows
Sensitive action planning	High-reliability model + validation	Conservative, auditable execution
Fallback	Secondary provider/model	Continuity during provider failure

12. Reasoning Engine

The Reasoning Engine assembles the ContextPackage, selects permitted tools, retrieves relevant memory/documents, calls the Model Router and streams grounded responses. It is the controlled thinking pipeline for MAIA.

ContextPackage contract

tenant, user, productContext, conversationSummary, recentMessages, relevantMemories, retrievedDocuments, structuredData, sensitiveSession, toolPolicy and costPolicy are assembled server-side before model call.

Grounding rule

The model receives only context that passed tenant, permission and sensitive-access gates.

13. Action Engine

The Action Engine turns recommendations into controlled execution. MAIA may propose actions freely, but execution requires permissions, confirmation policy, idempotency and audit logging.

Action Type	Allowed in v1	Confirmation
Create task	Yes	Depends on tenant policy
Generate report	Yes	No unless sensitive
Save memory	Yes	Sensitive/low-confidence requires review
Update project	Yes	Yes
Send email	Draft only v1	Yes before send
Generate quote	Draft only v1	Yes before send
Issue invoice	No in MVP	Always in future

Idempotency rule

Critical actions must include idempotencyKey to prevent duplicate invoices, emails, quotes or writes.

Tool Registry + Level 4 Operator Limits

Tool	Purpose	Permission	Confirmation	MVP
saveMemory	Create/update/archive governed memory.	memory.write	Sensitive/low confidence	Yes
createTask	Create operational task.	tasks.write	Tenant policy	Yes
updateProject	Modify project status/risk/summary.	projects.write	Yes	Yes
generateReport	Create executive report from allowed context.	reports.write	If sensitive	Yes
draftEmail	Draft email only, not send.	actions.propose	Before send later	Yes
createQuoteDraft	Generate quote draft.	quotes.write	Before final/send	Yes
readFiscalProfile	Read fiscal profile.	sensitive.read	Sensitive code	Orbit only
issueInvoice	Issue fiscal invoice.	sensitive.full_access	Always	No

Level 4 Rule	Definition
Can execute	Low/medium-impact actions inside allowed tenant and product context after permission checks.
Cannot execute	Irreversible, fiscal, legal, external-send or financial actions without explicit final confirmation.
Must log	Every execution attempt, success, failure and denial.
Must use idempotency	All critical or externally visible writes.
Must degrade safely	If permissions, tool result or model confidence are unclear, propose manual review instead.

Operational Boundaries

Allowed Actions

- Create tasks
- Update projects
- Generate reports
- Draft emails
- Draft quotes

Execute approved tools

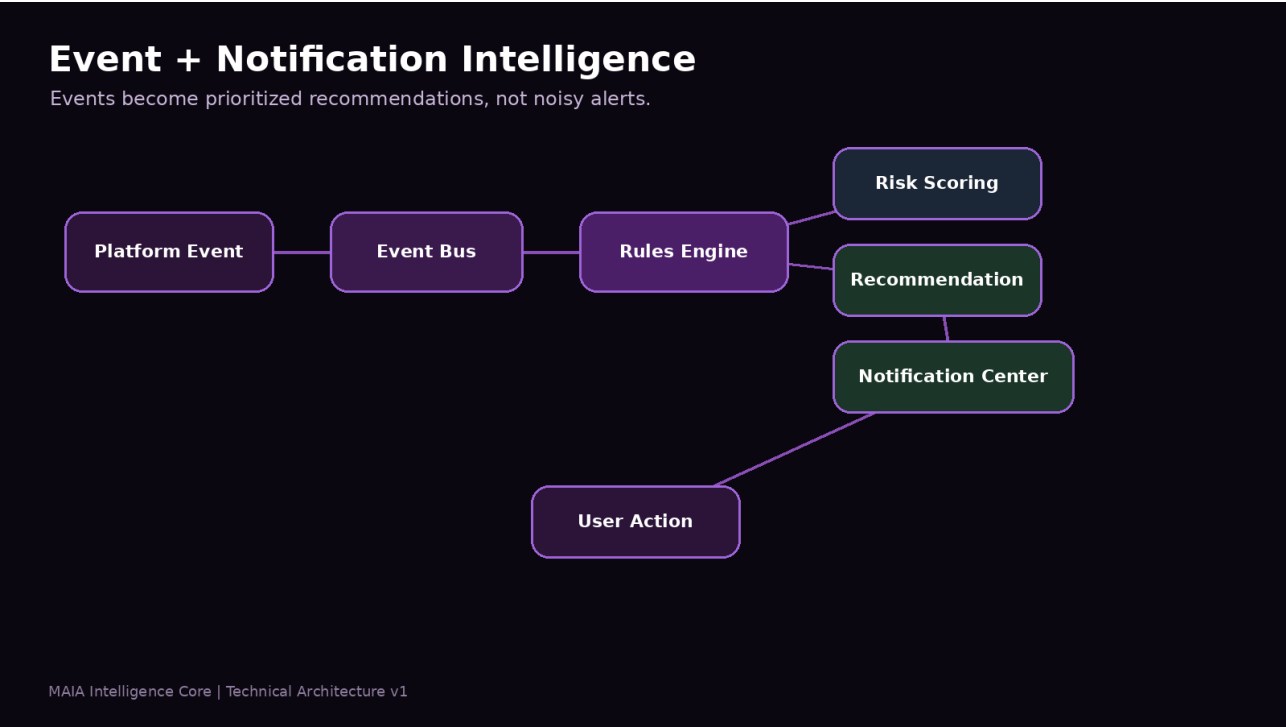
Restricted Actions

- Issue invoices automatically
- Modify fiscal identity
- Modify RFC
- Delete contracts
- Perform payments

Access sensitive data without verification

14. Event System

The Event System allows MAIA to react to platform changes. A document upload, task delay, invoice failure or cost threshold should become an event that can be evaluated and converted into a notification or recommendation.



Event Type	Example	Possible Outcome
document.uploaded	Contract uploaded	Queue analysis + notify
memory.conflict_detected	Fiscal data conflict	Review required
project.risk_detected	Due date close	Risk notification + recommendation
task.overdue	Missed due date	Action required
invoice.failed	Fiscal process failed	Review data
sensitive.access_failed	Failed code attempts	Security alert
cost.threshold_near	Usage cap near	Admin warning
tool.execution_failed	Action failed	Manual step recommendation

15. Notification Intelligence Engine

Notifications must be intelligent, not noisy. MAIA reads events, scores priority, identifies risk, creates recommendations and sends notifications only when they matter.

Priority	Meaning	Delivery Rule
INFO	Useful but not urgent	In-app only
WARNING	Needs attention soon	In-app + optional email digest
CRITICAL	Security/fiscal/operational risk	Immediate in-app + email
ACTION_REQUIRED	User decision needed	Pinned until resolved

Notification Scoring

Risk Score = Impact x Urgency x Confidence. Notify only when the score crosses the relevant threshold for the tenant/product. Do not notify every event. Notifications without judgment become spam with branding

Risk Score	Severity	Action
1-3	INFO	Store in notification center only.
4-6	WARNING	Notify in-app; include recommendation.
7-8	CRITICAL	Immediate in-app + email if enabled.
9-10	ACTION_REQUIRED	Pin until user resolves or delegates.

Notification Governance Rule

Only actionable events should generate notifications.
The goal is signal, not noise.

16. Cost Engine

The Cost Engine records model usage, embeddings, document processing, RAG searches, storage and tool executions by tenant, product and user. This prevents AI costs from becoming a surprise invoice with emotional damage attached.

Metric	Recorded In	Why It Matters
Input tokens	UsageLog	Request size/cost
Output tokens	UsageLog	Generation cost
Embedding calls	UsageLog	Indexing cost
Document chunks	DocumentProcessingLog	Processing load
RAG searches	UsageLog	Retrieval intensity
Storage bytes	StorageUsage	Storage growth
Tool executions	ToolLog/CostLedger	Operational usage
Estimated cost	CostLedger	Pricing and margins

Control	Meaning
Monthly budget	Tenant/product estimated cost budget for AI, storage and jobs.
Soft limit	Warn admins before budget is exceeded.
Hard limit	Block expensive operations except admin override.
Emergency stop	Disable AI calls/jobs for abusive or runaway usage.
Per-operation cap	Limit retrieved chunks, max tokens, file size and retry count.

Cost Governance

Control	Meaning	Default Value
Monthly Budget	Maximum estimated monthly cost for AI, storage, embeddings, jobs and integrations.	Tenant Defined
Soft Limit	Warn administrators before budget exhaustion.	80%
Hard Limit	Block non-essential expensive operations.	100%
Emergency Stop	Immediately disable AI calls and background jobs.	Enabled
Per-Operation Cap	Maximum cost allowed for a single operation.	Tenant Policy

Cost Protection Principles

Cost governance exists to prevent runaway automation, excessive AI consumption, uncontrolled storage growth and unexpected operational expenses.

Every tenant, product context and workflow must operate within predefined budget constraints.

MAIA should always prefer graceful degradation over uncontrolled spending.

Cost Escalation Policy

Budget Consumption	Severity	Action
0–79%	NORMAL	No action required
80–89%	WARNING	Notify administrators
90–99%	CRITICAL	Recommend optimization actions
100%+	HARD_LIMIT	Block expensive operations
Emergency Trigger	EMERGENCY	Suspend AI execution and background jobs

Cost Monitoring Scope

Category	Examples
AI Usage	OpenAI, Claude, Gemini requests
Embeddings	Vector generation and indexing
Storage	Documents, files, memory archives
Background Jobs	Queues, event processing, scheduled tasks
Integrations	External APIs and third-party services
Notifications	Email, messaging and delivery services

Cost Governance Rule

System stability has priority over unrestricted execution.

MAIA must never allow uncontrolled cost growth caused by automation, loops, malformed requests or excessive AI usage.

When predefined limits are reached, the platform should degrade gracefully and notify responsible users before service interruption occurs.

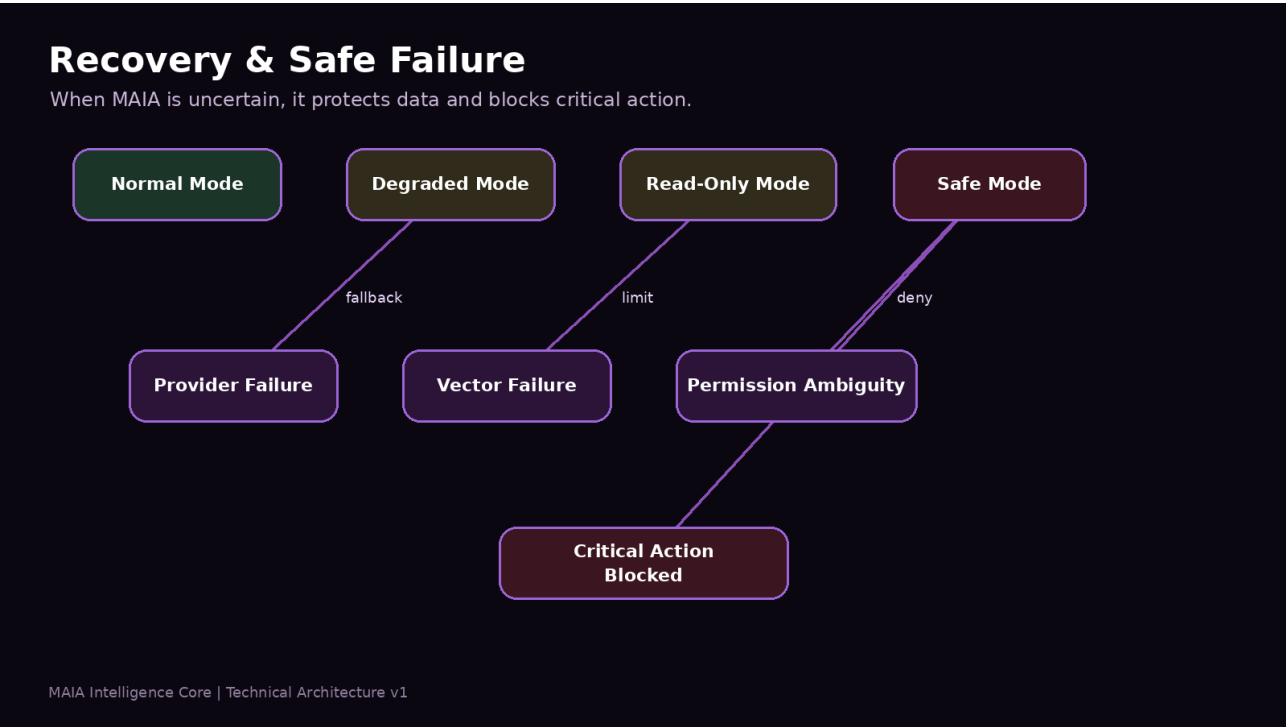
17. Audit & Explainability Layer

AITrace records evidence: model used, memories/documents retrieved, permissions evaluated, tools called, sensitive session status, cost estimate and confidence summary. It does not expose hidden chain-of-thought.

AITrace Field	Purpose
requestId	Connects response, tools, costs and audit
tenantId/userId	Scopes audit
productContext	Shows source product
modelProvider/modelName	Identifies route
memoryIdsUsed	Records memory evidence
documentChunkIdsUsed	Records RAG evidence
toolsCalled	Shows actions
permissionsEvaluated	Shows access decisions
sensitiveSessionUsed	Confirms sensitive use
confidenceSummary	Short evidence summary, not hidden reasoning
costEstimate	Usage analysis
fallbackUsed	Degraded behavior evidence

18. Backup & Recovery

MAIA must be recoverable. Backups, disaster recovery objectives and safe modes protect the system from provider failures, accidental deletion, corrupted data, bad deploys and the usual parade of software gremlins.



Area	Policy
Database backups	Daily full, hourly incremental
Retention	7 days hot, 30 days standard, 90 days archive
RPO	<= 1 hour
RTO	<= 4 hours
Recovery test	Quarterly restore test
Storage backup	R2 private storage with lifecycle rules and metadata integrity checks

Mode	Trigger	Allowed Behavior
Normal	All core services healthy	Full permitted capabilities
Degraded	Non-critical failure	Reduced context with limitation
Read-Only	Write path risk	Read allowed data, block mutations
Safe Mode	Security uncertainty	Block sensitive data and critical actions
Maintenance	Planned downtime	Disable actions and show status

19. Queue & Background Jobs

Long-running work must run in background jobs. Document extraction, chunking, embeddings, events and notifications should not block user-facing requests.

Job Type	Queue Use	Retry Policy
Document extraction	Async text extraction	3 retries then failed
Chunking/embeddings	Background indexing	Retry with backoff
Event processing	Evaluate event rules	Retry, then dead-letter
Notifications	Create/send alerts	Retry channel delivery
Cost aggregation	Periodic rollups	Retry nightly
Cleanup/retention	Scheduled deletion/archive	Log all results

20. Webhook Architecture

Webhooks allow Mikaelson, OFF, Orbit and future integrations to send or receive system events. Every webhook must use signatures, timestamps and replay protection.

Webhook Rule	Implementation
Endpoint format	/api/v1/webhooks/{source}
Signature	HMAC SHA-256 required
Replay protection	Timestamp window + eventId uniqueness
Retries	Exponential backoff
Idempotency	eventId stored and deduplicated
Payload logging	Store metadata, never raw sensitive payload unless encrypted

21. API Architecture

The API must be versioned, authenticated, tenant-scoped and permission-checked. Endpoints should be stable contracts between MAIA Core and connected products.

Endpoint	Purpose	Security
POST /api/v1/chat	Stream MAIA response	Auth + tenant + productContext
GET/POST /api/v1/conversations	List/create conversations	Authenticated
GET/POST /api/v1/memory	Read/create memory candidates	memory.read/write
PATCH /api/v1/memory/:id	Update/archive memory	memory.write + audit
POST /api/v1/documents/upload	Upload file	documents.upload
POST /api/v1/documents/:id/process	Queue extraction/chunking/embeddings	documents.process
POST /api/v1/sensitive/setup-code	Create/update code	Authenticated + verification
POST /api/v1/sensitive/verify-code	Create sensitive session	Lockout policy
POST /api/v1/actions/execute	Execute approved tool	actions.execute + confirmation
POST /api/v1/events	Internal event ingestion	System signed
GET/PATCH /api/v1/notifications	Read/resolve notifications	tenant-scoped
GET /api/v1/audit	Admin audit traces	admin.audit
GET /api/v1/costs	Usage/cost dashboard	admin.costs

API Standard	Decision
Versioning	All public/internal product APIs use /api/v1/*
Errors	Structured JSON with code, message, requestId
Rate limits	By user, tenant and operation
Idempotency	Required for critical write actions
Auth	Server-side validation through Clerk/session
Tenant guard	Every endpoint validates active tenant before business logic

API Contracts

POST /api/v1/chat

```
{ "tenantId": "tenant_x", "productContext": "mikaelson", "conversationId": "conv_x", "message": "What projects are at risk?", "clientState": { "currentPage": "/projects", "selectedProjectId": "proj_x" } }
```

Response stream events: token, tool_proposal, sensitive_access_required, notification_created, final, error.

POST /api/v1/sensitive/verify-code

```
{ "tenantId": "tenant_x", "code": "*****", "mode": "read | full" } -> { "sessionId": "ss_x", "mode": "read", "expiresAt": "ISO_TIMESTAMP" }
```

POST /api/v1/actions/execute

```
{ "tenantId": "tenant_x", "toolName": "createTask", "payload": {}, "confirmationToken": "confirm_x", "idempotencyKey": "idem_x" } -> { "status": "success | failed | pending", "toolLogId": "log_x" }
```

Contract Rule	Requirement
Versioning	All product-facing endpoints use /api/v1/*.
Errors	Return structured JSON with code, message, requestId and traceId where applicable.
Idempotency	Required for all critical writes, sends, quote actions and future fiscal actions.
Tenant Guard	tenantId is resolved and validated server-side before business logic.
Sensitive Gate	Sensitive data is blocked unless an active sensitive session exists.

API Contract Principles

All API contracts must remain backward compatible within the same major version.
Breaking changes require a new API version.

Every request must include:

- Authenticated user
- Tenant context
- Product context

Every response must include:

- Status
- Traceability information when applicable
- Structured error information when failures occur

Sensitive Access Rule - Sensitive Access Enforcement

Endpoints that expose sensitive information must require an active sensitive session.
Sensitive sessions are created through successful code verification and expire automatically after the configured timeout.
Sensitive access is never assumed from standard authentication alone.

22. File Processing Architecture

Files are uploaded to private object storage, validated, processed through queues, chunked, embedded and indexed. Large files must never be processed synchronously inside a user request.

File Concern	v1 Decision
Max PDF size	50 MB initial limit
Processing	Queue-based, never synchronous for large files
Validation	MIME/type/size scan before storing metadata
Storage	Cloudflare R2 private bucket
Extraction failure	Mark failed, notify user/admin
Security	Document chunks inherit parent permissions
Retention	Follows tenant/document policy

File Processing Rule

Files are never processed directly inside user-facing requests.
Large files must be processed asynchronously through the queue system.

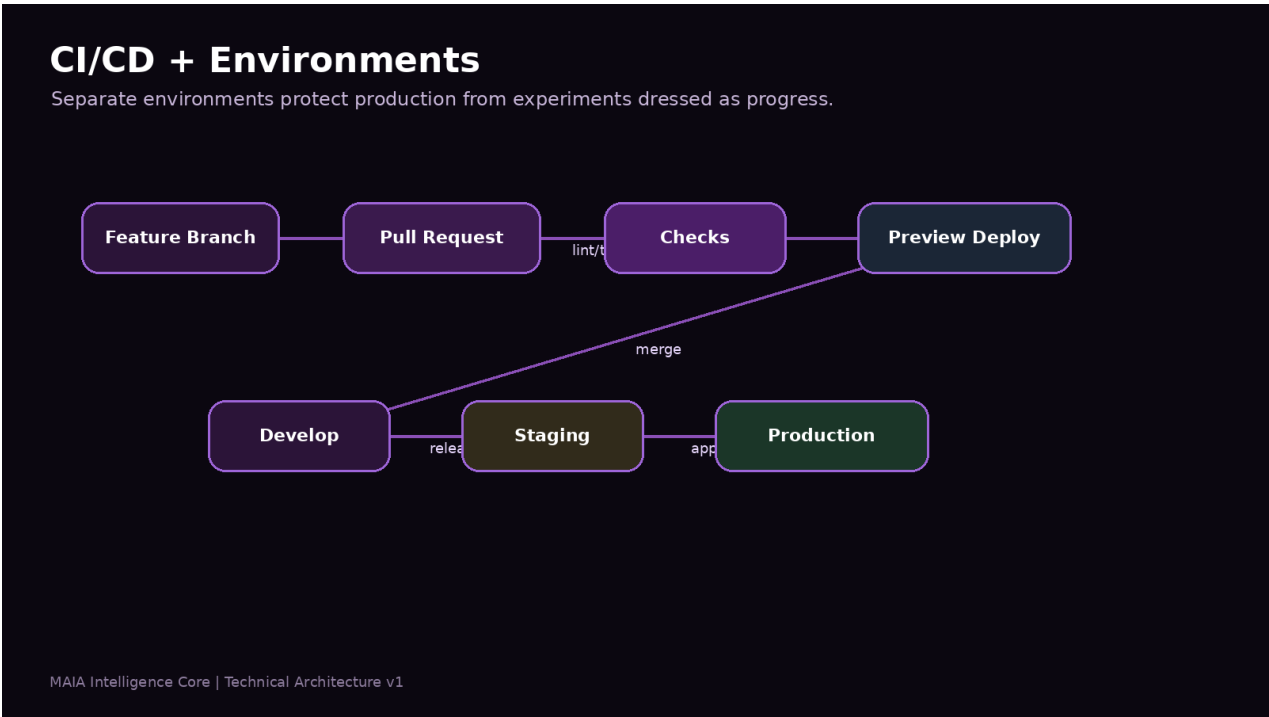
Every uploaded file receives:

- Tenant ownership
- Permission inheritance
- Processing status
- Audit trace

before becoming available to downstream AI workflows.

23. Environment Architecture

MAIA must separate local, development, staging and production environments. Secrets must never be committed to GitHub and production data must never be casually reused in development. Humanity has already suffered enough.



Environment	Purpose	Rules
Local	Developer machine	No production secrets/data
Development	Shared dev testing	Disposable data allowed
Preview	PR deployment	Isolated env vars
Staging	Release validation	Production-like config
Production	Live platform	Restricted deployments and secrets

Variable Group	Examples	Rule
Database	DATABASE_URL, DIRECT_URL	Separate per environment
Auth	CLERK_SECRET_KEY	Secret server-side only
AI	OPENAI_API_KEY	Only Model Router uses it
Storage	R2_*	Only storage service uses it
Security	ENCRYPTION_KEY, SENSITIVE_PEPPER	Rotate carefully
App	APP_URL, FEATURE_FLAGS	Controls staged releases

24. CI/CD Architecture

CI/CD defines how changes move safely from feature branches to production. The pipeline must include linting, type checks, tests, preview deployments, staging validation and rollback ability.

Stage	Gate
Feature branch	Developer work
Pull request	Review required
Checks	Lint, type check, unit/integration tests
Preview deploy	Vercel preview environment
Staging	Manual QA + migration review
Production	Approved deploy only
Rollback	Revert deployment and migration strategy documented

25. Observability & Monitoring

Observability answers: what failed, where, for whom, how much did it cost and how severe is it. MAIA needs logs, metrics, traces and alerts from the beginning.

Tool	Purpose
Sentry	Application errors and exceptions
PostHog	Product analytics and funnels
OpenTelemetry	Distributed traces
AITrace	AI-specific auditability
CostLedger	Usage and margin visibility
Status page later	Incident communication

Metric	Alert Condition
Error rate	Spike above baseline
Latency	Chat/API above threshold
AI cost	Tenant or product exceeds threshold
Sensitive failures	Multiple failed attempts
Queue backlog	Jobs delayed beyond SLA
Webhook failures	Repeated signature/retry failures

Monitoring Principle

Every critical user action, AI decision, tool execution and sensitive access event must be traceable.
If an event cannot be observed, measured or audited, it should be considered operationally invisible.

26. Testing Strategy

Testing should prioritize security, data isolation, sensitive access, RAG filtering, tool confirmation and recovery behavior. Pretty UI tests matter less than preventing cross-tenant leaks.

Test Type	Coverage
Unit	Permissions, memory merge, cost calc, model routing
Integration	APIs with tenant scoping and DB queries
Security	Cross-tenant denial, sensitive expiry, lockout
RAG	Permission inheritance and chunk filtering
Tool	Confirmation, idempotency, audit logs
Recovery	Provider/DB/storage failure
E2E	Invite user, chat, upload doc, ask question, notification

Testing Requirement

Security, tenant isolation and sensitive access tests are mandatory release gates.
A deployment cannot be promoted to production if any critical security, tenant isolation or sensitive access test fails.

27. MVP Scope

The MVP must build the foundation: multi-tenant auth, permissions, sensitive access, memory, documents, RAG, model router, action basics, events, notifications, audit, costs and admin visibility. Voice, WhatsApp, tax portal automation and Level 5 agents are excluded.

Included in MVP v1	Not Included in MVP v1
Multi-tenant auth with Clerk Organizations	Voice assistant
Roles and permissions	WhatsApp/Slack integrations
Sensitive Access Code and sessions	Real tax portal automation
Chat streaming with MAIA Core	Autonomous Level 5 agents
Executive Memory Engine	Native mobile app
Document upload, chunking, embeddings	Banking integrations
RAG retrieval with tenant permissions	Public marketplace
Model Router basic implementation	Advanced model benchmarking
Action Engine basic: tasks, memory, reports	Unconfirmed critical actions
Event Bus and in-app notifications	Complex workflow automation
AITrace, UsageLog, CostLedger	Enterprise SSO/SAML unless required

Sprint	Build Area	Deliverables
1	Foundation Layer	Next.js, Prisma, Clerk, tenant model, roles, permissions
2	Security Layer	Sensitive code, sessions, lockout, audit, masking
3	Memory Engine	CRUD, lifecycle, versions, merge/archive
4	Knowledge Engine	R2 upload, extraction, embeddings, pgvector
5	Reasoning Engine	Streaming, context assembly, model router
6	Action + Audit	Tool contracts, ToolLog, AITrace
7	Events + Notifications	Event model, rules engine, notification center
8	Admin Dashboard	Users, tenants, memory review, docs, usage, audit
9	Mikaelson Integration	First product integration

28. Development Standards

Development standards keep MAIA from becoming a beautiful pile of exceptions. Every module must respect tenant scope, security gates, service boundaries, auditability and explicit error handling.

- No direct model calls from UI. All AI calls go through Model Router.
- No database access without tenant scope unless SUPER_ADMIN path logs the action.
- No sensitive data in logs, prompts or traces unless masked/encrypted.
- No critical action without explicit confirmation.
- No deleting memory by default; archive first unless deletion is required.
- Every data-changing action must produce an audit trail.
- Every recommendation triggered by events must explain why it was triggered.
- Every endpoint must check auth, tenant and permissions before business logic.
- Every migration must be reviewed before production deployment.

Checklist Item	Required Before Sprint 1
Repository	Private GitHub repo named maia-intelligence
Auth	Clerk project with Organizations enabled
Database	Neon PostgreSQL ready for Prisma
Storage	Cloudflare R2 private bucket ready
AI	OpenAI key for Model Router v1
Environments	Local/dev/preview/staging/prod strategy
Security rule	No bypass of tenant guard, permissions or sensitive gate
MVP boundary	Voice/WhatsApp/tax automation/Level 5 excluded
First product target	Mikaelson OS after Core foundation

28. Architecture Readiness & Engineering Handoff

Architecture Readiness

Gate	Status After Inserts
Blueprint v1.1	Approved. Product/architecture vision is complete.
Technical Architecture v1	Approved for Sprint 1 after inserts.
Engineering Specification	Covered by inserted ERD, Prisma models, permissions, API contracts, scoring and tool registry.
Sprint 1	GO: Foundation Layer.
First Build Objective	Secure multi-tenant MAIA Core with auth, roles, permissions, sensitive access and data model.

Engineering Directive

No additional conceptual architecture documents should be created before Sprint 1.

The next engineering artifacts should be:

- Sprint 1 Build Specification
- Repository Setup
- Prisma Schema
- Initial Implementation Tasks

Hard Stop

Do not implement voice, WhatsApp, Gmail integrations, autonomous agents or fiscal automation during Sprint 1. The foundation layer must be completed before advanced capabilities are introduced.

Final CTO Instruction

The first engineering milestone is not “make MAIA answer.” The first milestone is “make MAIA answer inside the correct tenant, with the correct permissions, using allowed context, with auditability, cost visibility and safe failure behavior.” Once that exists, the intelligence layer can grow without turning into a beautiful liability.

Architecture status

This document is the consolidated MAIA Technical Architecture v1. It replaces fragmented technical notes and is the source of truth for Sprint 1 engineering execution.